

Kantonsschule Im Lee

Informatik

Daten & SQL

Skript

© Winterthur, 30. Januar 2024

Inhaltsverzeichnis

1	Vorwort	1
2	Daten & SQL	2
2.1	Einführung	2
2.1.1	Was sind Daten?	2
2.1.2	Datenmengen	2
2.1.3	Tabellen	3
2.2	Einzelne Tabellen abfragen mit Structured Query Language (SQL)	8
2.2.1	Syntax	8
2.2.2	Einfache Selektion: <code>SELECT</code>	8
2.2.3	Selektion mit Filter: <code>WHERE</code>	9
2.2.4	Eindeutige Selektion: <code>SELECT DISTINCT</code>	11
2.2.5	Ungefähre Treffer: <code>LIKE</code>	12
2.2.6	Sortieren: <code>ORDER BY</code>	14
2.2.7	Aggregatsfunktionen	14
2.2.7.1	<code>COUNT</code>	14
2.2.7.2	<code>MAX</code> , <code>MIN</code>	15
2.2.7.3	<code>SUM</code>	15
2.2.7.4	<code>AVG</code>	16
2.2.7.5	<code>LENGTH</code>	16
2.2.8	Gruppieren: <code>GROUP BY</code>	16
2.2.9	Erste / Letzte Zeilen: <code>LIMIT</code>	17
2.2.10	Verzweigungen: <code>CASE WHEN</code>	19
2.2.11	Anwendung: Personalisierte Werbung auf InstaHub	19
2.3	ERM	23
2.4	Mehrere Tabellen abfragen mit SQL	26
2.4.1	<code>INNER JOIN (=JOIN)</code>	26
2.4.2	<code>LEFT JOIN (=LEFT OUTER JOIN)</code>	28
2.4.3	<code>RIGHT JOIN (=RIGHT OUTER JOIN)</code>	29
2.4.4	<code>FULL JOIN (=FULL OUTER JOIN)</code>	29
2.5	Daten bearbeiten mit SQL	31
2.5.1	Eine neue Tabelle erstellen: <code>CREATE table</code>	31
2.5.2	Tabellen löschen: <code>DROP table</code>	32
2.5.3	Daten einfügen: <code>INSERT</code>	33
2.5.4	Daten verändern: <code>UPDATE</code>	34
2.5.5	Einträge löschen: <code>DELETE FROM</code>	34
2.6	Weiterführende Links	35

Kapitel 1

Vorwort

Dieses Skript befindet sich aktuell noch im Aufbau. Falls Sie Fehler finden oder Verbesserungen an der Struktur vorschlagen möchten, bin ich Ihnen dankbar für eine kurze Mail an cyril.wendl@edu.zh.ch.

Das Ziel dieses Skripts ist es, eine einheitliche und umfassende Dokumentation zu bieten, welche später bei Bedarf als Nachschlagewerk beigezogen werden kann.

Dieses Skript ist wie folgt gegliedert:

Aufgabe 1.1

Diese Boxen enthalten Aufgaben, die prüfungsrelevant sind, aber nicht abgegeben werden müssen.

Challenge-Aufgabe 1.2

Diese Boxen enthalten Aufgaben, die zur Vertiefung dienen.

Aufgabe (Abgabe) 1.3

Diese Boxen enthalten Aufgaben, die auf Moodle abgegeben werden sollten.

Achtung!

Diese Boxen enthalten wichtige Hinweise, um beispielsweise Datenverlust oder Fehler beim Programmieren zu vermeiden.

Kapitel 2

Daten & SQL

2.1 Einführung

2.1.1 Was sind Daten?

Der Begriff “Daten” ist der Plural von *Datum* und kommt aus dem Lateinischen (*datum* = gegeben, bzw. *dare* = geben). “Daten” bedeuten also im weitesten Sinne etwas “**Gegebenes**”, bzw. ein Faktum. Dies wiederum wirft die philosophische Frage auf, was ein Faktum ist. Beispiele von Fakten sind:

- “Mein Nachbar heisst Marco Odermatt.”
- “Aktuell findet an der KLU Unterricht statt.”
- “Das Billett von Zürich nach Winterthur kostet CHF 6.80.”
- “Das Logo der KLU besteht aus blauen und roten Farben.”
- ...

Anhand des letzten Beispiel sieht man, dass Fakten nicht unbedingt wahr sein müssen, um als Fakten zu gelten.

In der **Informatik** sind Daten so gut wie immer durch Nullen und Einsen repräsentiert. Also zum Beispiel:

- ...10100010101111011101001001...
- ...111011101110000111011010111...
- ...

Die Nullen und Einsen können beliebige Informationen darstellen, also beispielsweise Bilder, Texte, Videos oder Zahlen.

2.1.2 Datenmengen

Seit etwa einem guten halben Jahrhundert nehmen die Datenmengen stets zu. [Tabelle 2.1](#) zeigt die gängigen Grössenordnungen von Datenmengen auf.

Masseinheit	Dezimalsystem		Größenordnung
KB (Kilobyte)	10^3 B(yte)	1'000 B	Eine Text-Datei
MB (Megabyte)	10^6 B	1'000'000 B	Eine Musik-Datei
GB (Gigabyte)	10^9 B	1'000'000'000 B	Eine Video-Datei
TB (Terabyte)	10^{12} B	1'000'000'000'000 B	Kleiner Firmen-Server
PB (Petabyte)	10^{15} B	...	Facebook-Server
EB (Exabyte)	10^{18} B	...	Alle CERN-Daten
ZB (Zettabyte)	10^{21} B	...	Alle Daten (~ 100 ZB)
YB (Yottabyte)	10^{24} B	...	$\sim 2030?$

Tabelle 2.1: Gängige Daten-Größenordnungen in der Informatik (ein **Byte** = 8 bit)

Die gesamten weltweit vorhandenen Daten machen aktuell ca. 100 Zettabyte aus. Es wird erwartet, dass das erste Yottabyte gegen 2030 erreicht wird (s. [Abbildung 2.1](#)).

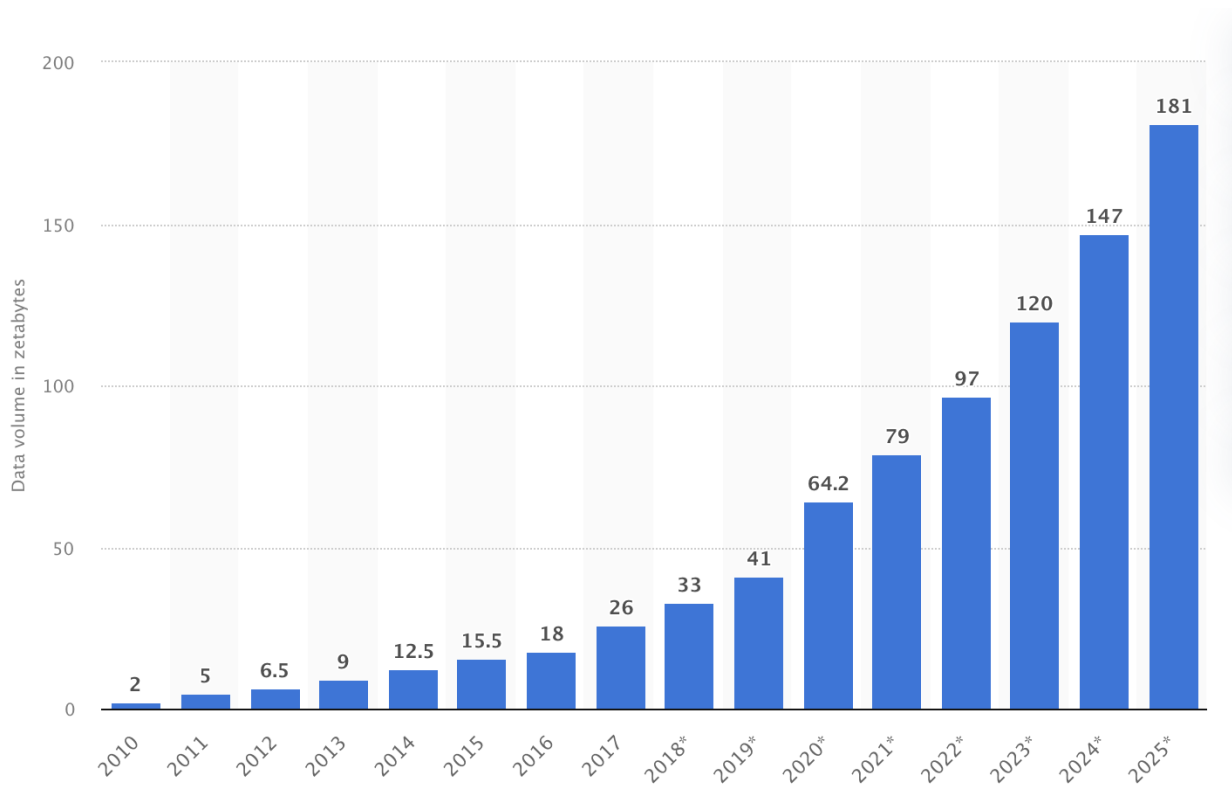


Abbildung 2.1: Entwicklung der globalen Datenmenge

Dies hat zur Folge, dass Datenzentren immer grösser werden und mehr Energie verbrauchen, wie das folgende Video verdeutlicht: https://youtu.be/_r97qdyQtIk?t=2m14s

2.1.3 Tabellen

Der allergrösste Teil der oben erwähnten Daten befindet sich in sogenannten Datenbanken. Datenbanken bestehen im Wesentlichen aus Tabellen, weshalb wir im Folgenden zunächst anschauen, wie eine Tabelle aufgebaut ist. Ein Beispiel einer Tabelle ist gegeben in [Tabelle 2.2](#).

Name	Region	Fläche	Einwohner	BIP
Afghanistan	Asien	652	25.8M	21.0B
Albanien	Europa	28.7	3.49M	5.6B
Algerien	Afrika	2,381.7	31.2M	147.6B
Amerikanische...	Ozeanien	0.199	65.4K	0.15B
Andorra	Europa	0.468	66.8K	1.2B
Angola	Afrika	1,246.7	10.1M	11.6B
Anguilla	Mittelamerika	0.091	11.8K	0.088B
Antarktik	Antarktis	14M	0	0
Antigua und...	Mittelamerika	0.442	66.4K	0.524B
Argentinien	Südamerika	2,766.9	36.9M	367B
...	...	...	...	...

Tabelle 2.2: Länder-Tabelle

Eine Tabelle besteht aus Zeilen und Spalten, wie in [Tabelle 2.3](#) gezeigt.

Schlüsselattribut (eindeutig!)

Spalte (=Kolonne, Attribut)

<u>Name</u>	Region	Fläche	Einwohner	BIP
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...

Zeile

Tabelle 2.3: Typische Tabellenstruktur anhand einer Tabelle mit Informationen zu Ländern

In [Tabelle 2.3](#) enthält jede **Zeile** Informationen zu einem Land: dessen Name, die Region (=Kontinent), Fläche, die Einwohnerzahl und das Bruttoinlandprodukt (BIP). Jede **Spalte** enthält Informationen eines selben Typs, beispielsweise die Regionen aller Länder, die Flächen oder die Einwohnerzahlen. In einer Spalte sollten also alle Werte denselben Typ haben, beispielsweise “Zahl” (Fläche, Einwohner, BIP) oder “Text” (Name, Region). Nicht zuletzt gibt es in gewissen Tabellen (genau) ein Schlüsselattribut, dessen Werte eindeutig sein müssen. Dies bedeutet in [Tabelle 2.3](#) beispielsweise, dass kein Ländername zweimal oder mehr vorkommen darf. Weshalb dies wichtig ist, werden wir später sehen.

Eine Datenbank ist vereinfacht gesagt eine Sammlung aus mehreren Tabellen, auf die mehrere Geräte zugreifen können (lesen und schreiben), wie in [Abbildung 2.2](#) gezeigt.

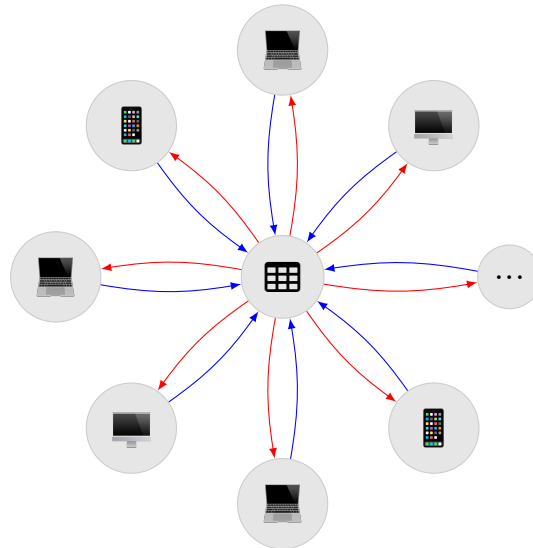


Abbildung 2.2: Geräte können Daten von der / zur Datenbank **senden** / **empfangen**

Datenbanken und Tabellen finden Anwendung in allen möglichen Bereichen des täglichen Lebens:

- **Social Media:** Instagram, TikTok, LinkedIn, etc.
- **Shopping:** Galaxus, AliExpress, etc.
- **Netzwerke:** SBB, swissgrid (Strom-Netzwerk), etc.

Wie können wir mit den riesigen Datenmengen umgehen und sinnvolle Einsichten daraus gewinnen?

Eine der verbreitetsten Arten, Tabellen und Datenbanken zu erstellen, zu verändern und zu analysieren ist die Programmiersprache **SQL**. Im Folgenden verwenden wir die Datenbanken aus der Lern-Plattform InstaHub, um uns mit **SQL** vertraut zu machen.

Alle Beispiele können direkt auf InstaHub ausgeführt werden. Die **Abgaben** sind rot markiert und befinden sich auf Moodle unter “Quiz **SQL**”.

 Aufgabe 2.1 Einrichtung von Instahub

	Selber eine Datenbank administrieren
	Einzelarbeit
	ca. 25 min.

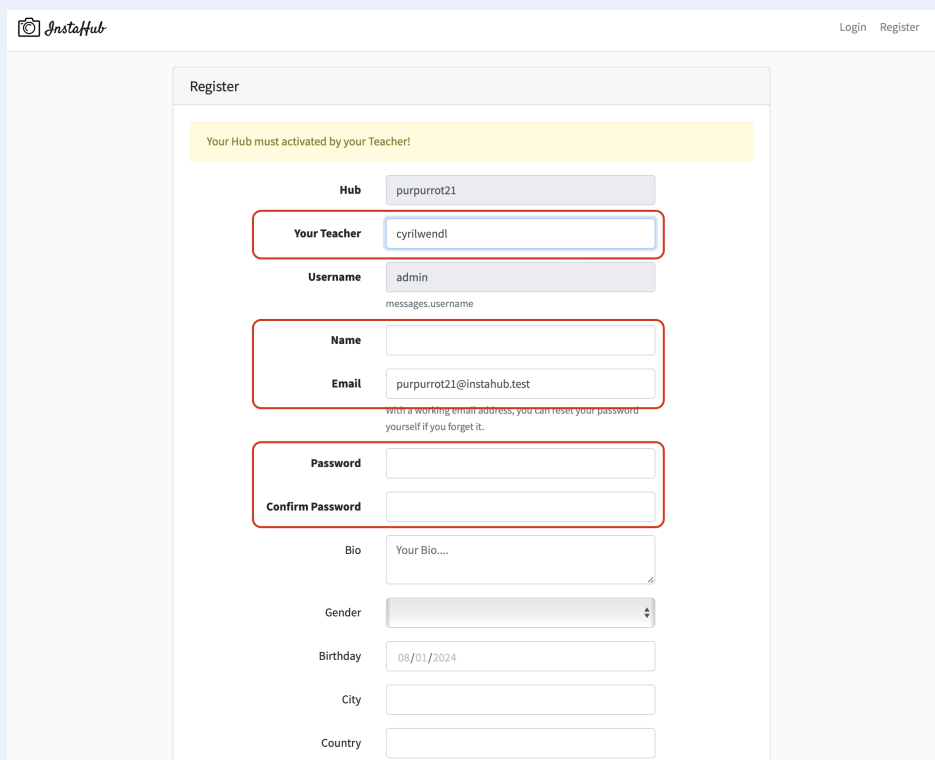
Führen Sie folgende Schritte aus:

1. Auf <https://instahub.org> gehen
2. Auf Hub erstellen (rechts oben) klicken und einen Hub erstellen.



3. Geben Sie folgende Angaben ein (s. Bild unten):

- Lehrername: **cyrilwendl**
- Username ist **admin** und kann nicht geändert werden.
- Ihren Namen und Ihr Passwort können Sie frei setzen. Das Passwort sollten Sie sich irgendwo notieren.
- Sie können entweder eine echte E-Mail-Adresse angeben oder die generierte E-Mail-Adresse so lassen. Im zweiten Fall können Sie jedoch Ihr Passwort nicht mehr per Mail zurücksetzen, falls Sie dieses je verlieren sollten.
- Geben Sie mir mündlich Bescheid, dass Sie die Seite erstellt haben. Ich muss nun Ihren Hub (=Ihre Webseite) manuell aktivieren. Dies dauert nur wenige Sekunden.



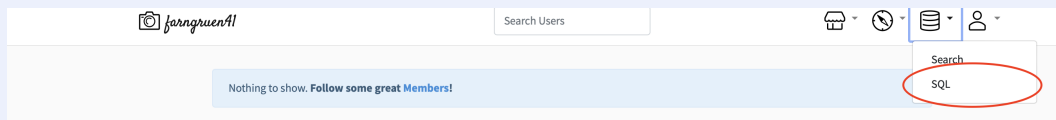
4. Sie können sich jetzt auf Ihrer individuellen Webseite (z.B. <https://purpurrot21.instahub.org>) mit dem Benutzernamen **admin** und dem von Ihnen zuvor gewählten Passwort anmelden. Sie sind nun **AdministratorIn** Ihres eigenen sozialen Netzwerks.

Aufgabe 2.2 Inbetriebnahme von Instahub

🏁	Lernen, mithilfe von SQL eigene Abfragen zu machen
👥	2er- bis 3er-Gruppen
🕒	ca. 60 min.

Bestimmen Sie eine Person pro Gruppe, die sich auf der eigenen Webseite (z.B. <https://purpurrot21.instahub.org>) mit dem Benutzernamen **admin** und dem entsprechenden Passwort anmeldet. Die zwei anderen Personen sollen das folgende Kapitel aus diesem Skript lesen und der ersten Person dabei helfen, die **SQL**-Befehle zu schreiben.

Sie können jetzt unter folgendem Menu-Punkt Ihre eigene Datenbank administrieren:



Als Erstes sehen Sie eine Liste der verfügbaren Tabellen sowie der dazugehörigen Spaltennamen (s. Bild unten). Tabellennamen sind in **orange**, Spaltennamen in **grün** markiert.

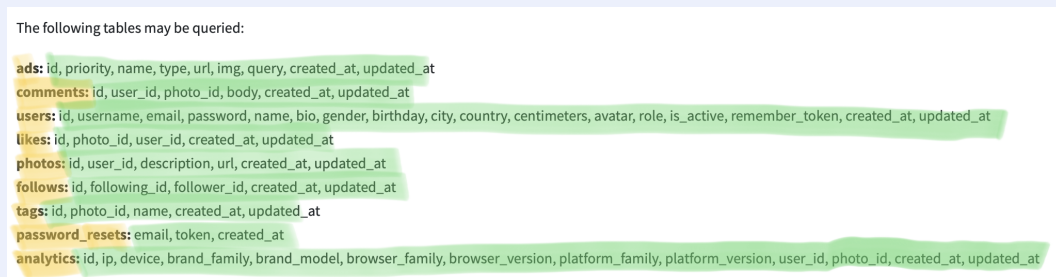


Abbildung 2.3: Die Tabellen von InstaHub und deren Attribute

2.2 Einzelne Tabellen abfragen mit SQL

2.2.1 Syntax

Die Sprache **Structured Query Language (SQL)** besteht, wie wir sehen werden, aus verschiedenen Schlüsselwörtern, wie etwa **SELECT**, **WHERE**, **GROUP BY**, **ORDER BY** usw. Dazu gibt es Folgendes zu beachten:

- Ob wir diese Schlüsselwörter gross oder klein schreiben, spielt keine Rolle. Allerdings wird empfohlen, die Schlüsselwörter immer in Grossbuchstaben zu schreiben, um sie von Kolonnen-Namen, Tabellen-Namen usw. abzugrenzen.
- Wir können alle **SQL**-Befehle auf einer einzigen Zeile schreiben, dadurch wird der Code aber deutlich weniger lesbar. Stattdessen empfiehlt es sich, vor gewissen Schlüsselwörtern wie **SELECT**, **WHERE** oder **GROUP BY** eine neue Zeile zu erstellen. Dazu lohnt es sich, die Beispiele genau anzuschauen.

Kommentare können in **SQL** auf zwei Arten geschrieben werden:

```
-- 1. Kommentar auf einer Zeile
/* 2. Kommentar über
mehrere Zeilen */
```

2.2.2 Einfache Selektion: **SELECT**

Mit dem Befehl “**SELECT** col1,col2,... **FROM** tablename” können wir gewisse Spalten (=Kolonnen) col1,col2,... aus einer Tabelle mit dem Namen tablename auswählen. Wir können statt den Spaltennamen auch einfach einen Stern (*) tippen, um alle Kolonnen einer Tabelle zu erhalten, d.h. die gesamte Tabelle.

Beispiel 2.3. Geben Sie auf InstaHub den folgenden Befehl ein (der **Kommentar** muss nicht kopiert werden, er dient lediglich zur Erklärung des Codes):

```
-- wähle alle Spalten aus der Tabelle users aus
SELECT *
FROM users
```

Mit diesem Befehl wählen wir alle Spalten (*) der Tabelle **users** aus. Sie sollten nun die gesamte Tabelle **users** sehen.

Aufgabe 2.4

Geben Sie alle Benutzernamen (**username**) aus **users** aus.

Lösungsvorschlag für Aufgabe 2.4

```
SELECT username
FROM users
```

Aufgabe 2.5

Geben Sie die Benutzernamen (**username**) und echten Namen (**name**) aller Einträge aus **users** aus.

✓ Lösungsvorschlag für Aufgabe 2.5

```
SELECT username, name
FROM users
```

Aufgabe 2.6

Geben Sie die Namen und Wohnorte aller Mitglieder aus.

✓ Lösungsvorschlag für Aufgabe 2.6

```
SELECT username, city
FROM users
```

2.2.3 Selektion mit Filter: **WHERE**

Fügen wir zu einer **SQL**-Abfrage den Befehl "**WHERE conditions**" hinzu, können wir das erhaltene Resultat nach gewissen Bedingungen (**conditions**) filtern.

Beispiel 2.7. Geben Sie folgendes Beispiel in Instahub ein:

```
SELECT name, city, gender
FROM users
WHERE gender="male"
```

Nebst dem Gleich-Zeichen (im obigen Beispiel) stehen folgende Filter-Operatoren zur Verfügung:

Operator in SQL	Mathematische Bedeutung
=	gleich (=)
<>	ungleich (≠)
<	kleiner als (<)
<=	kleiner oder gleich (≤)
>	grösser als (>)
>=	grösser oder gleich (≥)
BETWEEN x AND y	ein Wert zwischen x und y
IN (wert1, wert2, ...)	einer von mehreren möglichen Werten

Um mehrere Bedingungen miteinander zu verknüpfen, können wir die Befehle **AND**, **NOT** sowie **OR** verwenden.

Beispiel 2.8. Folgender Code gibt alle Mitglieder aus, die zwischen 172 und 174 cm gross sind.

```
SELECT name, centimeters, gender
```

```
FROM users
WHERE centimeters BETWEEN 172 AND 174
```

Beispiel 2.9. Folgender Code gibt alle den Namen und die Stadt aller Mitglieder aus, die entweder in Leipzig, Berlin oder Hamburg wohnen.

```
SELECT name, city
FROM users
WHERE city IN ("Leipzig", "Berlin", "Hamburg")
```

⚠ Achtung!

Wie Sie anhand von **Beispiel 2.9** und **Beispiel 2.8** sehen, spielt es in SQL eine Rolle, ob wir Zahlen oder Text filtern: Wenn nach einem Text gefiltert wird, muss dieser in Anführungszeichen stehen (wie z.B. **"Berlin"** in **Beispiel 2.9**). Bei Zahlen ist dies nicht nötig (s. z.B. die Zahl 172 in **Beispiel 2.8**).

✍ Aufgabe 2.10

Geben Sie alle weiblichen Mitglieder aus, die zwischen 150 und 155 gross sind.

✓ Lösungsvorschlag für Aufgabe 2.10

```
SELECT name, gender, centimeters
FROM users
WHERE gender="female"
AND centimeters BETWEEN 150 AND 155
```

✍ Aufgabe 2.11

Zeigen Sie den Namen, das Geburtsdatum sowie die Grösse aller Frauen an, die kleiner oder gleich 160 Zentimeter sind.

✓ Lösungsvorschlag für Aufgabe 2.11

```
SELECT name, birthday, centimeters
FROM users
WHERE gender="female"
AND centimeters <= 160
```

 Aufgabe 2.12

Verwenden Sie die Tabelle `comments`. und geben Sie alle Kommentare aus, die Entweder vom user mit der ID 10 oder vom user mit der ID 38 stammen.

 Lösungsvorschlag für Aufgabe 2.12

```
SELECT user_id, body
FROM comments
WHERE user_id IN (38, 10)
```

2.2.4 Eindeutige Selektion: `SELECT DISTINCT`

Der Befehl `DISTINCT` nach einem `SELECT` führt dazu, dass jeder mögliche Wert nur einmal ausgegeben wird, d.h. Duplikate werden entfernt.

 Aufgabe 2.13

Vergleichen Sie folgende zwei Befehle. Überlegen Sie sich zuerst, was der Code ausgeben sollte, bevor Sie die Befehle in InstaHub ausführen.

```
SELECT gender
FROM users
```

```
SELECT DISTINCT gender
FROM users
```

 Aufgabe 2.14

Geben Sie jeden Wohnort nur einmal aus.

 Lösungsvorschlag für Aufgabe 2.14

```
SELECT DISTINCT city
FROM users
```

 Aufgabe 2.15

Geben Sie jede Benutzerrolle nur einmal aus. `dba` bedeutet “Database Administrator”, also Datenbank-AdministratorIn.

 Lösungsvorschlag für Aufgabe 2.15

```
SELECT DISTINCT role
FROM users
```

2.2.5 Ungefähre Treffer: LIKE

Mit dem Befehl **LIKE** können wir die Tabelle nach Zeilen filtern, welche in einer gewissen Spalte ein bestimmtes Wort enthalten.

Beispiel 2.16. Mit folgendem Befehl erhalten wir alle Städte, die mit “Be...” beginnen:

```
SELECT DISTINCT city
FROM users
WHERE city LIKE "Be%"
```

Das “%”-Zeichen ist eine sogenannte *wildcard* und dient hier als Platzhalter, der Folgendes bedeutet: “es *kann* noch weiterer Text an dieser Stelle stehen, *muss* aber nicht” (s. [Tabelle 2.4](#)).

Symbol	Bedeutung
"%"	Stellt null oder mehrere, beliebige Zeichen dar
"_"	Stellt ein einzelnes, beliebiges Zeichen dar

Tabelle 2.4: *wildcards* in **SQL**

Beispiel 2.17. Mit folgendem Befehl erhalten wir alle Städte, die mit “H” beginnen und ein “m” an dritter Stelle haben:

```
SELECT DISTINCT city
FROM users
WHERE city LIKE "H_m%"
```

Aufgabe 2.18

Vergleichen Sie folgende drei Befehle. Überlegen Sie sich zuerst, was der Code ausgeben sollte, bevor sie die Befehle in InstaHub ausführen.

```
SELECT username, city
FROM users
WHERE city = "Berlin" AND name LIKE "Fabian%"
```

```
SELECT username, city
FROM users
WHERE city = "Berlin" OR name LIKE "Fabian%"
```

```
SELECT username, city
FROM users
WHERE city = "Berlin" AND NOT name LIKE "Fabian%"
```

 Aufgabe 2.19

Finden Sie alle Berliner die Marc heissen.

☒ Lösungsvorschlag für Aufgabe 2.19

```
SELECT name, city
FROM users
WHERE city="Berlin" AND name LIKE "Marc%"
```

 Aufgabe 2.20

Finden Sie alle Linas und Lorenas.

☒ Lösungsvorschlag für Aufgabe 2.20

```
SELECT name, city
FROM users
WHERE name LIKE "Lina%"
OR name LIKE "Lorena%"
```

 Challenge-Aufgabe 2.21

Überprüfen Sie, welche Mitglieder im Jahr 2001 geboren sind (mit der Spalte `birthday`).

☒ Lösungsvorschlag für Aufgabe 2.21

```
SELECT name, birthday
FROM users
WHERE birthday LIKE "2001%"
```

 Aufgabe (Abgabe) 2.22

Wählen Sie alle Personen mit dem Namen *Naomi* aus, die nicht aus Berlin kommen.

☒ Lösungsvorschlag für Aufgabe 2.22

```
SELECT name, city
FROM users
WHERE name LIKE "Naomi%"
AND city <> "Berlin"
```

🏆 Challenge-Aufgabe 2.23

Geben Sie die Grösse und den Wohnort von Juliette Amsel sowie Juliette Unger aus. Sie sollten das Zeichen "%" nicht verwenden.

✓ Lösungsvorschlag für Aufgabe 2.23

```
SELECT name, centimeters, city
FROM users
WHERE name LIKE "Juliette ___e_"
```

2.2.6 Sortieren: ORDER BY

Mit dem Befehl `ORDER BY col ASC` oder `ORDER BY col DESC` kann das Resultat einer **SQL**-Abfrage nach einer bestimmten Kolonne `col` sortiert werden. Mit `ASC` wird das Resultat aufsteigend und mit `DESC` absteigend sortiert.

Beispiel 2.24. Beobachten Sie das Resultat folgender Abfrage:

```
SELECT DISTINCT city
FROM users
ORDER BY city DESC
```

📝 Aufgabe 2.25

Geben Sie alle Benutzernamen in sortierter Reihenfolge aus ($a \rightarrow z$).

✓ Lösungsvorschlag für Aufgabe 2.25

```
SELECT username
FROM users
ORDER BY username ASC
```

2.2.7 Aggregatsfunktionen**2.2.7.1 COUNT**

Mit dem Befehl `COUNT` kann man die Anzahl Resultate zählen.

```
SELECT COUNT(*)
FROM users
```

Wir können den berechneten Wert zudem zur besseren Lesbarkeit umbenennen, indem wir den Befehl `AS` verwenden:

```
SELECT COUNT(*) AS "Registrierte Mitglieder"
FROM users
```


 Aufgabe 2.26

Geben Sie die Anzahl registrierten Mitglieder in Berlin aus. Die resultierende Spalte soll "Registrierte Mitglieder in Berlin" heissen.

 Lösungsvorschlag für Aufgabe 2.26

```
SELECT COUNT(*) AS "Registrierte Mitglieder in Berlin"
FROM users
WHERE city="Berlin"
```

2.2.7.2 MAX, MIN

Um die höchsten, bzw. tiefsten Werte einer Kolonne zu erhalten, können die Befehle **MIN** und **MAX** verwendet werden:

Beispiel 2.27. Beobachten Sie das Resultat folgender Abfrage:

```
SELECT MAX(centimeters) AS "Maximale Körpergrösse"
FROM users
```

 Aufgabe 2.28

Zeigen Sie, wie gross das kleinste Mitglied ist.

 Lösungsvorschlag für Aufgabe 2.28

```
SELECT MIN(centimeters) AS "Kleinste Körpergrösse"
FROM users
```

 Aufgabe 2.29

Zeigen Sie, wann sich zuletzt ein Mitglied registriert hat.

 Lösungsvorschlag für Aufgabe 2.29

```
SELECT max(created_at) AS "Zuletzt Registriert"
FROM users
```

2.2.7.3 SUM

Mit **SUM** kann die Summe einer Datenserie ausgegeben werden.

Beispiel 2.30. Folgender Code berechnet die Summe der Körpergrössen aller Personen, die Felix heissen:

```
SELECT SUM(centimeters)
FROM users
```

```
WHERE name LIKE "Felix%"
```

2.2.7.4 AVG

Mit **AVG** kann der Durchschnitt einer Datenserie ausgegeben werden.

Beispiel 2.31. Folgender Code berechnet die durchschnittliche Körpergrößen aller Personen, die Felix heissen:

```
SELECT AVG(centimeters)
FROM users
WHERE name LIKE "Felix%"
```

2.2.7.5 LENGTH

Mit **LENGTH** gibt die Anzahl Zeichen eines Texts zurück.

Beispiel 2.32. Folgender Code berechnet die Länge aller Namen:

```
SELECT name, LENGTH(name)
FROM users
```

2.2.8 Gruppieren: GROUP BY

Um die Resultate einer Anfrage pro Untergruppen zu sehen, kann der Befehl **GROUP BY** verwendet werden.

Beispiel 2.33. Folgender Code gibt die Anzahl Mitglieder pro Stadt aus:

```
SELECT city, COUNT(*) AS "Mitglieder pro Stadt"
FROM users
GROUP BY city
```

Zudem ist es möglich, Daten auch in Untergruppen zusammenzufassen.

Beispiel 2.34. Folgender Code gibt die Anzahl männlicher und weiblicher Mitglieder pro Stadt aus:

```
SELECT city, gender, COUNT(*)
FROM users
GROUP by city, gender
```

Achtung!

Falls nach einem **GROUP BY** die Resultate noch weiter eingegrenzt werden soll, muss statt **WHERE** der Befehl **HAVING** verwendet werden. Der Befehl **WHERE** wird verwendet, um die ursprünglichen Daten *vor* einem **GROUP BY** zu filtern, der Befehl **HAVING** wird verwendet, um die Resultate *nach* einem **GROUP BY** zu filtern.

↑ Aufgabe (Abgabe) 2.35

Geben Sie die durchschnittliche Körpergrösse aller Mitglieder in jeder Stadt aus. Zeigen Sie nur die Städte, in denen die Menschen im Durchschnitt zwischen 150 und 155 gross sind.

✓ Lösungsvorschlag für Aufgabe 2.35

```
SELECT city, AVG(centimeters) AS "Durchschnittliche Körpergrösse"
FROM users
GROUP BY city
HAVING `Durchschnittliche Körpergrösse` BETWEEN 150 AND 155
```

🏆 Challenge-Aufgabe 2.36

Geben Sie die maximale Körpergrösse aus, gruppiert nach Stadt und Geschlecht, für alle Städte, die mit dem Buchstaben "B" beginnen

✓ Lösungsvorschlag für Aufgabe 2.36

```
SELECT city, gender, MAX(centimeters) AS "Grösste Körpergrösse"
FROM users
WHERE city LIKE "B%"
GROUP BY city, gender
```

2.2.9 Erste / Letzte Zeilen: LIMIT

Mit dem Befehl **LIMIT** kann das Resultat einer **SQL**-Abfrage auf eine gewisse Anzahl Zeilen beschränkt werden.

Beobachten Sie das Resultat folgender Abfrage:

📝 Aufgabe 2.37

Zeigen Sie nur 3 Mitglieder (nur deren Namen) an.

✓ Lösungsvorschlag für Aufgabe 2.37

```
SELECT name
FROM users
LIMIT 3
```

 Aufgabe 2.38

Geben Sie die drei Städte mit den meisten Mitgliedern an.

☒ Lösungsvorschlag für Aufgabe 2.38

```
SELECT city, COUNT(*) AS "Anzahl Mitglieder"
FROM users
GROUP BY city
ORDER BY `Anzahl Mitglieder` DESC
LIMIT 3
```

 Aufgabe (Abgabe) 2.39

Zeigen Sie die Namen und Körpergröße der 5 grössten Mitglieder an.

☒ Lösungsvorschlag für Aufgabe 2.39

```
SELECT name, centimeters
FROM users
ORDER BY centimeters DESC
LIMIT 5
```

 Challenge-Aufgabe 2.40

Geben Sie die Stadtnamen aus, wo die meisten Mitglieder mit einem “b” im Namen wohnen (nur die ersten drei Zeilen).

Tipps:

- Berechnen Sie zuerst mit einem **GROUP BY**-Befehl die Anzahl Einwohner pro Stadt
- Beschränken Sie danach Ihre Abfrage mit dem **WHERE**-Befehl auf Benutzer, die ein “n” im Namen haben
- Sortieren Sie dann nach der Anzahl Mitglieder
- Verwenden Sie den **LIMIT**-Befehl erst am Schluss

☒ Lösungsvorschlag für Aufgabe 2.40

```
SELECT city, COUNT(*) AS "Mitglieder"
FROM users
WHERE name LIKE "%b%"
GROUP by city
ORDER BY Mitglieder
DESC LIMIT 3
```

2.2.10 Verzweigungen: CASE WHEN

Der SQL-Befehl **CASE WHEN** kann helfen, Ausdrücke basierend auf Bedingungen zu schreiben, also ähnlich wie if-elif-else-Verzweigungen in Python.

Beispiel 2.41. Die durchschnittliche Körpergrösse in Deutschland beträgt 179 cm für Männer. Folgender Code berechnet für alle Männer, ob sie grösser, kleiner oder gleich dem Durchschnitt sind, und gibt das Resultat in einer neuen Kolonne aus.

```
SELECT name, centimeters,
CASE
  WHEN centimeters > 179 THEN 'Gross'
  WHEN centimeters < 179 THEN 'Klein'
  ELSE 'Genau im Durchschnitt'
END AS Durchschnittlich
FROM users
WHERE gender="male"
```

↑ Aufgabe (Abgabe) 2.42

Berechnen Sie eine neue Kolonne, die den Text “langer Name” enthält, falls ein Name länger als 20 Zeichen lang ist, und ansonsten “kurzer Name”. BenutzerInnen mit langem Namen sollen zuoberst stehen. Zeigen Sie nur die ersten 5 Zeilen.

✓ Lösungsvorschlag für Aufgabe 2.42

```
SELECT name, LENGTH(name),
CASE
  WHEN LENGTH(name)>17 THEN "Langer Name"
  ELSE "Kurzer Name"
END AS "Langer oder kurzer Name"
FROM users
ORDER BY `Langer oder kurzer Name` DESC
LIMIT 5
```

2.2.11 Anwendung: Personalisierte Werbung auf InstaHub

Auf InstaHub können Werben an verschiedene User angepasst werden. Dafür sind auf InstaHub noch einige weitere Tabellen angelegt, deren **Entity-Relationship Model (ERM)** in **Abbildung 2.4** abgebildet ist.

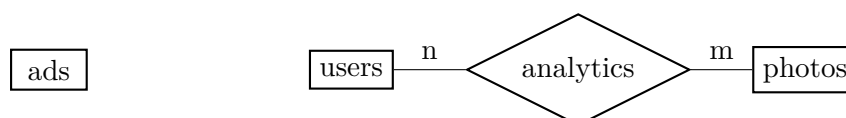


Abbildung 2.4: **ERM** für den Werbe-Teil von InstaHub

Mit der Tabelle **analytics** beginnt InstaHub das Verhalten der Besucher zu überwachen. Dabei wird der Besuch von Photo-Detailansichten mit folgenden Werten dokumentiert:

Feld	Beschreibung
<code>id</code>	Primärschlüssel, fortlaufende Nummer
<code>ip</code>	Die ersten drei Blöcke der IPv4-Adresse
<code>device</code>	desktop, mobile, tablet oder bot
<code>brand_family</code>	wird oft nur bei Smartphones mitgesendet, etwa Apple oder Samsung
<code>brand_model</code>	wird oft nur bei Smartphones mitgesendet, etwa GALAXY S5
<code>browser_family</code>	oft Firefox, Chrome oder Safari
<code>browser_version</code>	Die Versionsnummer des Browsers
<code>platform_family</code>	Das Betriebssystem, oft Windows, Mac, GNU/Linux, iOS oder Android
<code>platform_version</code>	Die Versionsnummer der Plattform
<code>user_id</code>	Benutzer der sich das Foto angesehen hat
<code>photo_id</code>	Angesehenes Foto
<code>created_at</code>	Zeitpunkt, als das Foto sich angesehen wurde
<code>updated_at</code>	i.d.R. wie <code>created_at</code> , nur anders, wenn manuell geändert

Tabelle 2.5: Kolonnen der Tabelle `analytics`

Mit diesen Informationen, sowie weiteren Informationen (Hashtags, Geschlecht, Wohnort etc.) kann InstaHub gezielt personalisierte Werbung schalten. Nach diesem Prinzip funktionieren auch andere soziale Medien wie TikTok, Instagram oder Snapchat.

Auf InstaHub kann können Werbeanzeigen mit **SQL** personalisiert werden. Dafür geht man zunächst auf die Werbungs-Seite (s. [Abbildung 2.5](#))

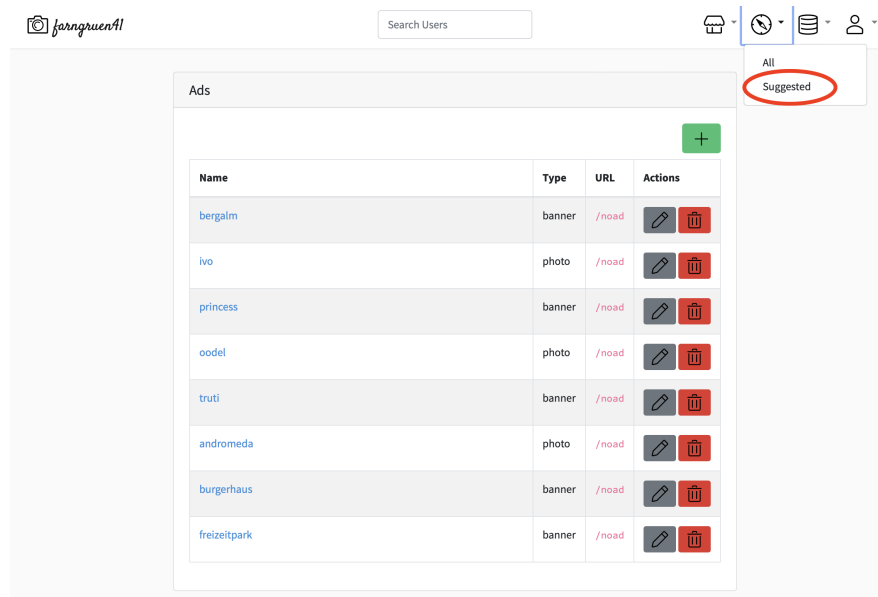


Abbildung 2.5: Übersicht der existierenden Werbungs-Kampagnen

Von dort aus kann man auf eine Werbekampagne klicken, um mehr Details zu sehen. Die Detail-Ansicht der ersten Kampagne ist abgebildet auf [Abbildung 2.6](#).

The screenshot shows a web application interface for editing a campaign named 'bergalm'. The form has the following fields:

- Name:** bergalm
- Type:** Banner (with a dropdown arrow)
- Priority:** 1 (with a note: "If two or more Ads are possible the one with the lowest Number (e.g. 1) will be chosen.")
- URL:** /noad (with a note: "Link of your Campaign.")
- Image:** /img/ad/bergalm.jpg (with a note: "Absolut or relative Uri to Image wich will be shown.")
- SQL-Query:** SELECT CASE WHEN description LIKE '%natur%' OR description LIKE '%landschaft%' OR description LIKE '%berg%' THEN true ELSE false

Below the form is a **Preview** section showing a banner image with the text "Wanderspaß in der Eifel" over a landscape photo.

Abbildung 2.6: Beispielkampagne

Als erstes sehen wir den (frei wählbaren) Kampagnen-Namen **bergalm**. Wir sehen auch dass die Werbung vom Typ “banner” ist, sie wird also bei Fotos angezeigt. Nebst weiteren Informationen sehen wir insbesondere zuunterst die “SQL-Query”, also die SQL-Abfrage, mit der eine Werbung personalisiert wird. Für diese Natur-Werbung haben wir folgende Abfrage:

```
SELECT CASE
  WHEN description LIKE '%natur%'
  OR  description LIKE '%landschaft%'
  OR  description LIKE '%berg%' THEN true
  ELSE false
END
FROM photos
WHERE id=$photo
```

Diese Abfrage wird nun auf der Webseite für jedes Foto ausgeführt, um zu sehen, ob die Werbung zum Foto passt.

Wir sehen zuerst, dass die Kolonne **description** der Tabelle **photos**, also die Kolonne, die Hash-tags speichert, auf Texte wie “Natur”, “Landschaft” und “Berg” abgesucht wird. Falls so ein Text vorhanden ist in den Hashtags des Fotos, wird der Wert **true** zurückgegeben, und somit wird die Werbung potentiell angezeigt, andernfalls wird der Wert **false** zurückgegeben. Auf der letzten Zeile sehen wir eine Variable **\$photo**, mit welcher jedes Foto der gesamten Webseite nach den genannten Kriterien abgesucht werden kann.

 Aufgabe 2.43

Verändern Sie den SQL-Befehl so, dass die Werbung auch für Fotos mit dem Hashtag “#Wasser” angezeigt werden. Testen Sie danach, ob es geklappt hat, indem Sie Fotos mit diesem Hashtag suchen. Sie können auf folgender URL überprüfen, ob es geklappt hat: [\[ihrefarbe\].instahub.org/p/1517](https://ihrefarbe.instahub.org/p/1517).

 Lösungsvorschlag für Aufgabe 2.43

```
SELECT CASE
  WHEN description LIKE '%natur%'
  OR  description LIKE '%landschaft%'
  OR  description LIKE '%wasser%'
  OR  description LIKE '%berg%' THEN true
  ELSE false
END
FROM photos
WHERE id=$photo
```

 Challenge-Aufgabe 2.44

Erstellen Sie eine neue Werbekampagne (s. [Abbildung 2.5](#)).

 Challenge-Aufgabe 2.45

	Gemeinsam ein soziales Netzwerk verwalten
	3er- bis 4er-Gruppen
	ca. 25 min.

Um sich ein Konto auf einer Seite einer anderen Person zu erstellen:

- Bestimmen Sie jemanden als [AdministratorIn](#) und verwenden Sie nur die Webseite dieser Person, also z.B. <https://purpurrot21.instahub.org>. Die anderen Personen sind [BenutzerInnen](#).
- [BenutzerInnen](#): Gehen Sie auf die von der Administratorin erstellte Webseite (<https://namedesnetzwerks.instahub.org>, wobei “namedesnetzwerks” ersetzt werden muss) und erstellen Sie ein neues Konto für sich.
- [AdministratorIn](#): Die neuen Accounts der [BenutzerInnen](#) muss dann von Ihnen als [AdministratorIn](#) aktiviert werden, indem Sie den User suchen, auf “Bearbeiten” und dann ganz unten auf “Account ist freigeschaltet” klicken.
- [AdministratorIn](#) und [BenutzerIn](#): Posten Sie nun falls Sie möchten Bilder in den sozialen Netzwerken Ihrer Kollegen, vergeben Sie Likes, folgen Sie anderen und kommentieren Sie fleissig in den anderen Netzwerken (max. 5 Minuten).
- Erstellen Sie nun personalisierte Werbungen für Ihre KlassenkameradInnen (s. [Abbildung 2.5](#)).

2.3 ERM

Ein **Entity-Relationship Model (ERM)** ist eine abstrakte Darstellung der Tabellen (Entities), die in einer Datenbank angelegt sind, sowie der Beziehungen (Relationships) zwischen den Tabellen. Jede Tabelle kann mehrere Kolonnen (=Attribute) haben.

Wenn man eine Datenbank erstellt, kann es nützlich sein, zuerst ein **ERM** aufzuzeichnen, um sich zu überlegen, welche Tabellen es braucht.

Das **ERM** des sozialen Netzwerks InstaHub ist in **Abbildung 2.7** abgebildet. **Abbildung 2.8** zeigt die Darstellung der Komponenten eines ERM in der sogenannten Chen-Notation.

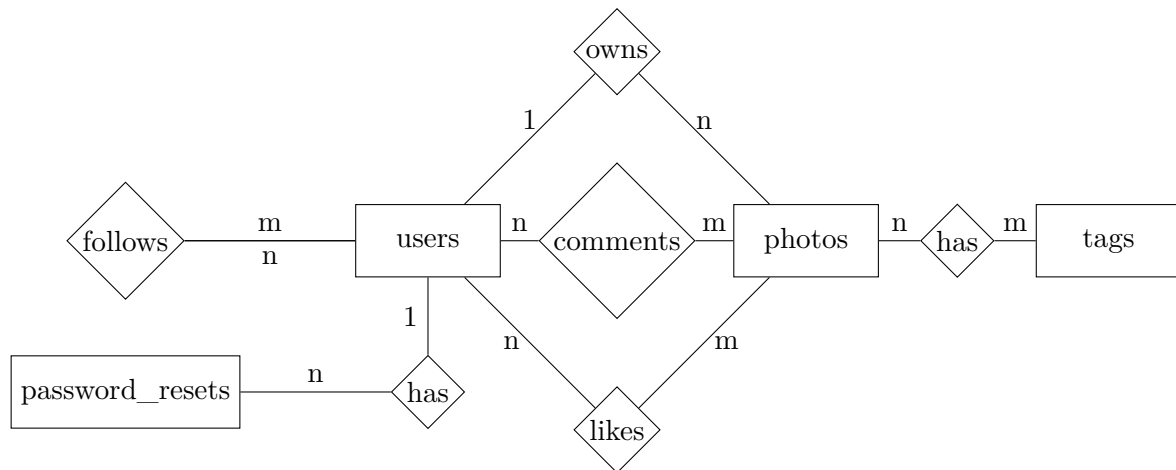


Abbildung 2.7: **ERM** von InstaHub

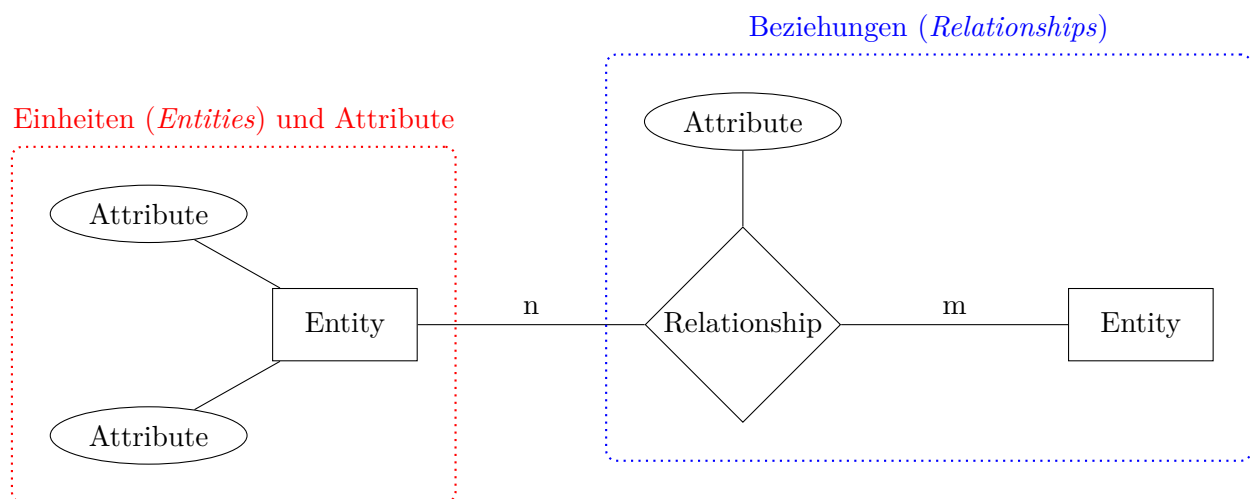


Abbildung 2.8: **ERM**-Komponenten in der sogenannten Chen-Notation

Die Entitäten, die meistens einer Tabelle entsprechen, werden als Rechtecke angegeben. Beziehungen stellen “Verknüpfungen” zwischen verschiedenen Tabellen dar und sind meist als Raute dargestellt. Die Attribute werden typischerweise als Kreise angegeben. In **Abbildung 2.7** sind die Attribute einfachheitshalber nicht angegeben.

Die Zahlen über den Strichen bezeichnen die **Kardinalität** der Beziehungen: Mit der Kardinalität wird ausgedrückt, wie viele Entitäten mit einer anderen Entität in Verbindung stehen können oder

müssen:

- **Eins-zu-Eins (1:1)**: Dies wird normalerweise angezeigt, indem eine “1” in der Nähe beider Entitäten platziert wird, die durch eine Beziehung verbunden sind. Es bedeutet, dass eine Instanz einer Entität mit genau einer Instanz einer anderen Entität assoziiert ist.
- **Eins-zu-Viele (1:n)**: Dies wird dargestellt, indem eine “1” in der Nähe der Entität auf der ‘eins’-Seite der Beziehung und ein “n” oder “m” (für “*many*”=viele) in der Nähe der Entität auf der “viele”-Seite der Beziehung platziert wird. Es zeigt an, dass eine Instanz der ersten Entität mit null, einer oder mehreren Instanzen der zweiten Entität assoziiert sein kann, aber eine Instanz der zweiten Entität nur mit einer Instanz der ersten Entität assoziiert sein kann.
- **Viele-zu-Viele (m:n)**: Dies wird gezeigt, indem ein “m” oder “n” in der Nähe beider Entitäten platziert wird. Es zeigt an, dass Instanzen der ersten Entität mit null, einer oder mehreren Instanzen der zweiten Entität assoziiert sein können und umgekehrt.

Folgende Fakten können beispielsweise am ERM in **Abbildung 2.7** abgelesen werden:

- Jeder **user** kann beliebig viele (oder 0) **comments** schreiben.
- Jeder **user** kann beliebig vielen (oder 0) anderen Usern folgen und kann von beliebig vielen (oder 0) andern usern gefollowt werden.
- Jedes Foto kann beliebig viele (oder 0) Tags haben. Jeder Tag kann auf n Fotos angewandt werden.
- Jedes Fotos gehört genau zu einem Benutzerprofil (**owns**). Jedes Benutzerprofil kann beliebig viele (oder 0) Fotos veröffentlichen.
- Jeder **user** kann sein Passwort beliebig viele male (oder 0 mal) zurückgesetzt haben. Jedes des **password_resets** kann aber nur einen **user** betreffen

Aufgabe 2.46

Eine Schule möchte ihre Datenbankstruktur neu organisieren, um Informationen über Lehrer, Schüler und Klassen besser verwalten zu können. Die Schule hat entschieden, ein Entity-Relationship-Modell (ERM) zu entwerfen, um die Beziehungen zwischen diesen Entitäten klar darzustellen (s. **Abbildung 2.9**). Was fällt Ihnen in diesem ERM auf? Stimmen die Kardinalitäten?

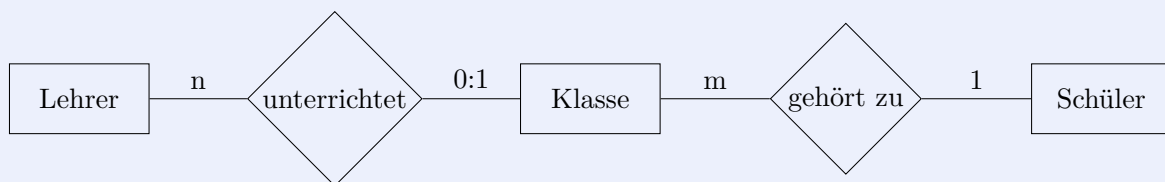


Abbildung 2.9: ERM einer Schul-Datenbank

Lösungsvorschlag für Aufgabe 2.46

Ein Lehrer kann in diesem ERM nur 0 oder 1 Klasse unterrichten. In Realität ergibt es keinen Sinn, dass ein Lehrer keine Klasse unterrichtet, und auch die Beschränkung auf maximal eine Klasse ergibt keinen Sinn. Statt 0:1 sollte also m stehen zwischen “unterrichtet” und “Klasse”.

Aufgabe 2.47

Ein Krankenhaus möchte seine Datenbankstruktur neu organisieren, um Informationen über Ärzte, Patienten und Behandlungen effizienter zu verwalten. Das Krankenhaus hat sich entschieden, ein Entity-Relationship-Modell (ERM) zu entwerfen, um die Beziehungen zwischen diesen Entitäten klar darzustellen (s. [Abbildung 2.10](#)).

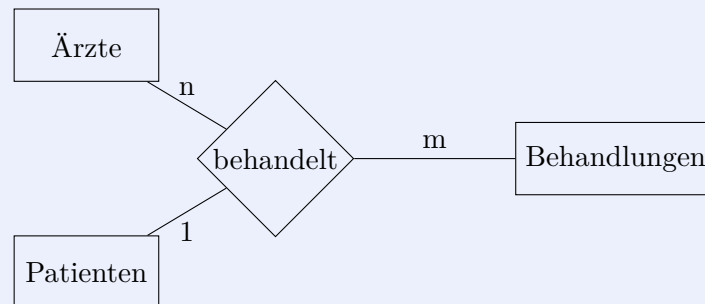


Abbildung 2.10: ERM eines Krankenhaus-Datenbanksystems

Bewerten Sie das dargestellte ERM. Sind die Kardinalitäten korrekt angegeben?

✓ Lösungsvorschlag für Aufgabe 2.47

In dem dargestellten ERM wird angenommen, dass ein Arzt mehrere Behandlungen durchführen kann (n:m-Beziehung) und dass jeder Patient genau eine Behandlung erhält (1:m-Beziehung). Diese Annahme ist jedoch nicht ganz realistisch, da ein Patient mehrere Behandlungen erhalten könnte. Die Kardinalität zwischen "Patienten" und "behandelt" sollte daher m sein, um anzugeben, dass ein Patient mehrere Behandlungen erhalten kann.

2.4 Mehrere Tabellen abfragen mit SQL

Häufig kann es nützlich sein, mehrere Tabellen miteinander zu verbinden, um neue Einsichten in die Daten zu gewinnen. Verschiedene Tabellen-Verbindungen sind schematisch in [Abbildung 2.11](#) abgebildet. Die Bedeutung dieser Abbildungen wird in den folgenden Abschnitten erklärt.



Abbildung 2.11: Venn-Diagramme unterschiedlicher SQL-Joins

2.4.1 **INNER JOIN (=JOIN)**

Die Syntax eines **INNER JOIN**-, oder auch einfach **JOIN**-Befehls sieht wie folgt aus:

```
SELECT kolonne1, kolonne2, ...
FROM tabelle1
INNER JOIN tabelle2
ON tabelle1.kolonne_id1 = tabelle2.kolonne_id2
```

Hier verbinden wir zwei Tabellen, indem wir zwei Kolonnen `kolonne_id1` und `kolonne_id2`, die derselben Information entsprechen (beispielsweise die user-ID), miteinander abgleichen. Beim **INNER JOIN** werden nur die Zeilen aus `tabelle1` retourniert, für die es einen entsprechenden Wert in `kolonne_id2` in `tabelle2` gibt. Alle Werte in `tabelle1` die mit keinem Wert in `tabelle2` überschneiden, sowie umgekehrt, tauchen nicht im Resultat auf (s. [Abbildung 2.12](#)).

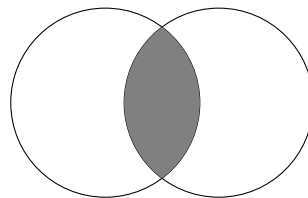


Abbildung 2.12: (**INNER**) **JOIN**

Wichtig zu wissen ist zudem, dass `kolonne1`, `kolonne2` usw. auf der ersten Zeile nun sowohl von `tabelle1` wie auch `tabelle2` stammen können.

Beispiel 2.48. Folgender Code gibt die ids aller user zurück, denen der user Mika Kaufmann folgt:

```
SELECT follows.following_id, follows.follower_id, users.name
```

```
FROM follows JOIN users
ON follows.follower_id = users.id
WHERE name="Mika Kaufmann"
```

Wie Sie sehen, kann es hilfreich sein, im **SELECT**-Ausdruck explizit die Tabelle zu nennen, von welcher eine bestimmte Kolonne stammt.

Zudem können auch mehr als zwei Tabellen miteinander verbunden werden.

Aufgabe 2.49

Geben Sie die ids aller Fotos in der Tabelle **likes** an, die der user Mika Kaufmann gelikt hat.

Lösungsvorschlag für Aufgabe 2.49

```
SELECT photo_id, user_id, name
FROM likes
INNER JOIN users
ON likes.user_id=users.id
WHERE name="Mika Kaufmann"
```

Challenge-Aufgabe 2.50

Challenge: Erstellen Sie eine Liste, wo für jedes Hamburger Mitglied die Anzahl seiner Fotos aufgeführt ist. Die Liste soll in absteigender Reihenfolge die Anzahl Fotos pro Mitglieder auflisten.

Lösungsvorschlag für Aufgabe 2.50

```
SELECT city, username, COUNT(*) AS "Anzahl Fotos"
FROM photos
JOIN users ON photos.user_id = users.id
WHERE city="Hamburg"
GROUP BY username
ORDER BY `Anzahl Fotos` DESC
```

 Aufgabe 2.51

Es soll Werbung an alle Strandurlauber verschickt werden. Finden Sie alle Photos die den Hashtag **meer** enthalten. Geben Sie den Namen, die Emailadresse, den Geburtstag und die Stadt der zugehörigen Benutzer aus.

 Lösungsvorschlag für Aufgabe 2.51

```
SELECT description, name, email, birthday, city
FROM photos
JOIN users on photos.user_id = users.id
WHERE description LIKE "%#meer%"
```

 Aufgabe (Abgabe) 2.52

Geben Sie die 5 User mit den meisten Followern aus.

 Lösungsvorschlag für Aufgabe 2.52

```
SELECT name, COUNT(*) AS "Followers"
FROM users JOIN follows on follows.following_id = users.id
GROUP BY name
ORDER BY Followers DESC
LIMIT 5
```

 Challenge-Aufgabe 2.53

Finden Sie heraus, welche 5 Fotos am meisten Likes erhalten haben. Geben Sie den Benutzernamen und Namen der Ersteller der Fotos an, die id der Fotos sowie die Anzahl Likes.

 Lösungsvorschlag für Aufgabe 2.53

```
SELECT users.username, users.name AS "Name", photos.id AS "Photo id",
       COUNT(likes.photo_id) AS "Likes"
FROM users
JOIN photos ON photos.user_id = users.id
JOIN likes ON photos.id = likes.photo_id
GROUP BY users.name, likes.photo_id
ORDER BY Likes DESC
LIMIT 5
```

2.4.2 LEFT JOIN (=LEFT OUTER JOIN)

Ein **LEFT JOIN**, oder **LEFT OUTER JOIN** unterscheidet sich von einem **INNER JOIN** dadurch, dass **alle** Einträge der ersten Tabelle (nach dem **SELECT**-Befehl) in der neuen Tabelle erscheinen. Werte aus der zweiten Tabelle, die nicht in der ersten sind, werden jedoch nicht angezeigt (s. [Abbildung 2.13](#)).

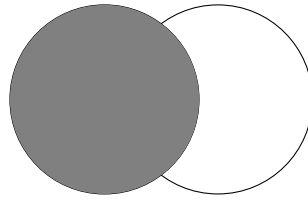


Abbildung 2.13: LEFT (OUTER) JOIN

Aufgabe 2.54

Diejenigen BenutzerInnen, die noch keine Fotos hochgeladen haben, sollen per Email dazu aufgefordert werden, Fotos hochzuladen. Finden Sie alle users, die noch keine Fotos hochgeladen haben. Geben Sie deren Name und Email-Adresse aus. Verwenden Sie statt **WHERE** den Befehl **HAVING**.

Lösungsvorschlag für Aufgabe 2.54

```
SELECT users.name, users.email, COUNT(photos.id) AS "Anzahl Fotos"
FROM users
LEFT JOIN photos ON photos.user_id = users.id
GROUP BY users.name
HAVING `Anzahl Fotos` = 0
```

2.4.3 RIGHT JOIN (=RIGHT OUTER JOIN)

Ein **RIGHT JOIN** funktioniert analog zu einem **LEFT JOIN** (s. [Abbildung 2.14](#)).

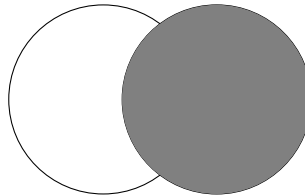


Abbildung 2.14: RIGHT (OUTER) JOIN

2.4.4 FULL JOIN (=FULL OUTER JOIN)

Bei einem **FULL (OUTER) JOIN** werden alle Werte aus der ersten und der zweiten Tabelle angezeigt (s. [Abbildung 2.15](#)).

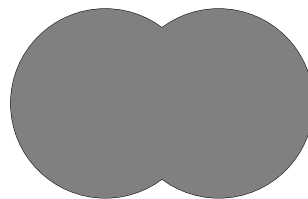


Abbildung 2.15: FULL (OUTER) JOIN

In InstaHub funktioniert der **FULL (OUTER) JOIN** nicht, weshalb es an dieser Stelle keine Übungen hierzu gibt.

2.5 Daten bearbeiten mit SQL

Mit SQL kann man Daten nicht nur abfragen, sondern auch erstellen, verändern und löschen. Im folgenden Schauen wir uns dafür nötigen Befehle an.

2.5.1 Eine neue Tabelle erstellen: `CREATE table`

Eine neue Tabelle kann innerhalb einer Datenbank mit dem Befehl `CREATE TABLE` angelegt werden:

```
CREATE TABLE tabellenname(
    kolonne1 eigenschaften1,
    kolonne2 eigenschaften2,
    ...
)
```

Beispiel 2.55. Die Tabelle `users` wurde mit folgendem Befehl erstellt:

```
CREATE TABLE users (
    `id` int(10) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
    `username` varchar(191) NOT NULL UNIQUE,
    `email` varchar(191) NOT NULL UNIQUE,
    `password` varchar(191) NOT NULL,
    `name` varchar(191) NOT NULL,
    `bio` varchar(191) DEFAULT NULL,
    `gender` enum('male','female') DEFAULT NULL,
    `birthday` datetime DEFAULT NULL,
    `city` varchar(191) DEFAULT NULL,
    `country` varchar(191) DEFAULT NULL,
    `centimeters` int(11) DEFAULT NULL,
    `avatar` varchar(191) NOT NULL DEFAULT 'avatar.png',
    `role` enum('user','dba','teacher','admin') NOT NULL DEFAULT 'user',
    `is_active` tinyint(1) NOT NULL DEFAULT 0,
    `remember_token` varchar(100) DEFAULT NULL,
    `created_at` timestamp NULL DEFAULT NULL,
    `updated_at` timestamp NULL DEFAULT NULL
)
```

Nach dem Kolonnenamen folgt zuerst der **Kolonnentyp**, d.h., die Angabe, welche Art von Daten in der Kolonne gespeichert werden. Einige der häufigsten Datentypen sind:

- `INT(laenge)` bezeichnet eine Ganzzahl, für die `laenge` bytes zur Verfügung stehen. Beispielsweise speichert die Kolonne `centimeters` eine Zahl von 0 bis 2^{11} , d.h. 2048.
- `TINYINT(1)` speichert den Wert 0 oder 1, wobei 0 typischerweise “falsch” und 1 “wahr” bedeuten.
- `DATETIME` ist ein Datum mit einer Uhrzeit
- `VARCHAR(laenge)` bezeichnet eine Zeichenkette, d.h. ein Text, der mit maximal `laenge` vielen Bit kodiert wird.
- `ENUM('wert1', 'wert2', ...)` bezeichnet eine Auflistung (engl. *enumeration*) möglicher Werte. Beispielsweise können nur “male” und “female” im Feld `gender` eingetragen werden.

Zudem können folgende Eigenschaften für Kolonnen angegeben werden:

- **NOT NULL**: Feld darf nicht leer sein. Falls kein **DEFAULT**-Wert angegeben wird, muss, der Wert für diese Kolonne beim Erstellen eines neuen Benutzerkontos immer mitgeliefert werden.

- **DEFAULT** wert: Standard-Wert, falls nichts anderes angegeben wird. Das Attribut `avatar` nimmt beispielsweise den Wert `avatar.png` an, falls kein anderes Bild hochgeladen wird.
- **AUTO_INCREMENT**: Zahl, die bei jedem neuen Eintrag automatisch immer um 1 grösser wird
- **PRIMARY KEY**: Schlüsselattribut (s. [Tabelle 2.3](#)).
- **UNIQUE**: Darf keine doppelten Werte enthalten

2.5.2 Tabellen löschen: **DROP table**

Mit dem Befehl **DROP TABLE** `tabellenname` kann eine Tabelle komplett löschen.

Achtung!

Dieser Schritt ist irreversibel. Falls Sie eine Tabelle irrtümlich löschen, geben Sie mir Bescheid, damit ich Ihre Datenbank zurücksetzen kann.

Aufgabe 2.56

Erstellen Sie eine Tabelle `users2` mit denselben Eigenschaften wie `users` (s. [Beispiel 2.55](#))

Lösungsvorschlag für Aufgabe 2.56

```
CREATE TABLE users2 (  
  `id` int(10) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,  
  `username` varchar(191) NOT NULL UNIQUE,  
  `email` varchar(191) NOT NULL UNIQUE,  
  `password` varchar(191) NOT NULL,  
  `name` varchar(191) NOT NULL,  
  `bio` varchar(191) DEFAULT NULL,  
  `gender` enum('male','female') DEFAULT NULL,  
  `birthday` datetime DEFAULT NULL,  
  `city` varchar(191) DEFAULT NULL,  
  `country` varchar(191) DEFAULT NULL,  
  `centimeters` int(11) DEFAULT NULL,  
  `avatar` varchar(191) NOT NULL DEFAULT 'avatar.png',  
  `role` enum('user','dba','teacher','admin') NOT NULL DEFAULT 'user',  
  `is_active` tinyint(1) NOT NULL DEFAULT 0,  
  `remember_token` varchar(100) DEFAULT NULL,  
  `created_at` timestamp NULL DEFAULT NULL,  
  `updated_at` timestamp NULL DEFAULT NULL  
)
```

Aufgabe 2.57

Löschen Sie danach die Tabelle `users2` wieder.

Lösungsvorschlag für Aufgabe 2.57

```
DROP TABLE users2
```

2.5.3 Daten einfügen: INSERT

Mit **INSERT** können neue Zeilen in eine bestehende Tabelle eingefügt werden.

Die Syntax von **INSERT** sieht wie folgt aus:

```
INSERT INTO tabelle_name (kolonne1, kolonne2, kolonne3, ...)
VALUES (wert1, wert2, wert3, ...)
```

Beispiel 2.58. Folgender Code fügt eine neue Person in die Tabelle `users` ein:

```
INSERT INTO users (username, email, password, name, bio, gender, birthday, city,
country, centimeters, avatar, role, is_active, remember_token, created_at,
updated_at)
VALUES ('guenther37', 'guenther@instahub.test', '12345', 'Günther Müller', '
Günther mag Kartoffelsalat.', 'male', '2006-06-06 00:00:00', 'Leipzig', '
Deutschland', '173', 'avatar.png', 'user', '0', NULL, now(), now())
```

Wie Sie sehen, wird zuerst die Tabelle (`users`) aufgelistet, danach die Kolonnen und schliesslich die Werte.

Aufgabe 2.59

Für welche Kolonnen müssen für einen neuen Benutzer immer die Werte mitgegeben werden?
S. die Erklärungen zu **NOT NULL** und **DEFAULT** in [Abschnitt 2.5.1](#).

Lösungsvorschlag für Aufgabe 2.59

```
username, email, password, name, role, is_active
```

Aufgabe (Abgabe) 2.60

Erstellen Sie einen neuen Eintrag in der Tabelle `users` mit folgenden Eigenschaften:

Kolonne	Wert
username	testuser
email	test@test.com
passwort	test123
name	Test-Vorname Test-Nachname
role	user
is_active	1

Überprüfen Sie danach, ob der Eintrag korrekt erstellt worden ist, indem Sie **SELECT * FROM users** ausführen. Ihr(e) Tisch-Nachbar(in) soll nun versuchen, sich mit diesen Angaben auf Ihrer InstaHub-Seite einzuloggen.

Lösungsvorschlag für Aufgabe 2.60

```
INSERT INTO users (username, email, password, name, role, is_active)
VALUES ('testuser', 'test@test.com', 'test123', 'Test-Vorname Test-
Nachname', 'user', '1')
```

2.5.4 Daten verändern: UPDATE

Mit **UPDATE** können Werte einer Tabelle aktualisiert werden. Die Syntax von **UPDATE** sieht wie folgt aus:

```
UPDATE tabelle_name
SET kolonne1 = wert1, kolonne2=wert2, ...
WHERE bedingung(en)
```

Aufgabe 2.61

Ersetzen Sie die Stadt “Berlin” überall durch “Bern”

☒ Lösungsvorschlag für Aufgabe 2.61

```
UPDATE users
SET city="Bern"
WHERE city="Berlin"
```

Aufgabe (Abgabe) 2.62

Setzen Sie die Körpergrößen aller männlichen Mitglieder auf 190.

☒ Lösungsvorschlag für Aufgabe 2.62

```
UPDATE users
SET centimeters=190
WHERE gender="male"
```

2.5.5 Einträge löschen: DELETE FROM

Mit dem Befehl **DELETE FROM** können Einträge aus einer Tabelle gelöscht werden.

```
DELETE FROM tabellenname
WHERE bedingungen
```

Beispiel 2.63. Folgender Code löscht den Eintrag für user **guenther37**:

```
DELETE FROM users
WHERE username="guenther37"
```

Aufgabe 2.64

Löschen Sie den Eintrag, den Sie in **Aufgabe 2.56** erstellt haben. Überprüfen Sie, ob der Eintrag verschwunden ist, indem Sie danach **SELECT * FROM users** ausführen.

✓ Lösungsvorschlag für Aufgabe 2.64

```
DELETE FROM users
WHERE username="testuser"
```

2.6 Weiterführende Links

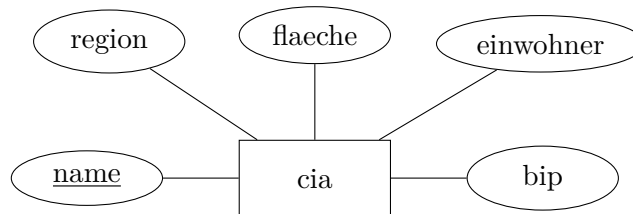
Folgende Links dienen der Vertiefung in das Thema **SQL** (und der Prüfungsvorbereitung).

Allgemein zu Datenbanken:

- <https://oinf.ch/kurs/vernetzung-und-systeme/datenbanken/>
- <https://wi-wissen.github.io/instahub-doc-de/#/exercices>

Zu **SQL**:

- <https://www.w3schools.com/sql/default.asp> (auf Englisch)
- <https://sql-island.informatik.uni-kl.de>
- <https://sql-tutorial.de/home/lektionen.php?lektion=1>. Hier wird unter anderem folgende Datenbank verwendet:



Akronyme

ERM	Entity-Relationship Model
SQL	Structured Query Language